Static-Analysis-Guided LLM Repair of Quantum Programs

Proyecto de Ingeniería de Software Cuántico

Ingeniería de Software + Computación Cuántica

2026

Idea central del proyecto

Pregunta simple

¿Puede un LLM reparar mejor bugs en programas cuánticos si recibe evidencia de análisis estático?



La contribución no es crear un nuevo LLM ni un nuevo lenguaje cuántico, sino estudiar si la **evidencia de Ingeniería de Software** mejora la reparación automática.

Contexto: ¿por qué importa?

Software cuántico

Programas escritos con frameworks como `Qiskit` combinan:

- código Python clásico;
- circuitos cuánticos;
- qubits, gates y mediciones;
- simuladores y backends.

Problema de calidad

Los errores pueden venir de:

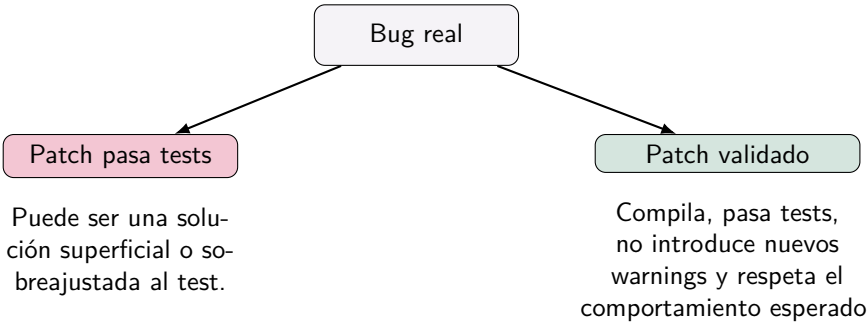
- mal uso de APIs;
- operaciones cuánticas incorrectas;
- mediciones mal ubicadas;
- incompatibilidades de versión;
- pruebas insuficientes.

Motivación

La comunidad ya estudia testing, depuración, análisis estático y reparación automática, pero todavía falta entender **qué evidencia ayuda realmente a reparar bugs cuánticos**.

El problema en palabras simples

Un LLM puede generar un parche, pero eso no garantiza que el parche sea bueno.



El proyecto distinguirá entre **patch plausible** y **patch validado**.

Qué ya existe y qué falta

Línea existente	Qué aporta	Qué deja abierto
Bugs4Q	Bugs reales de programas Qiskit con versiones buggy/fixed y tests.	No propone un método de reparación.
LLM repair	Estudios iniciales muestran que LLMs pueden reparar algunos bugs cuánticos.	Falta saber qué tipo de evidencia mejora la reparación.
LintQ / QChecker	Detectan patrones sospechosos mediante análisis estático.	Detectar no es reparar; los warnings no siempre explican el bug.
Mutation-guided repair	Usa mutantes para guiar la reparación.	No se centra en evidencia estática específica de QSE.

Brecha de investigación

No está suficientemente claro si los **warnings de análisis estático cuántico** pueden transformarse en evidencia útil para que un LLM genere mejores parches.

Novedad del proyecto

Estudios previos	**Este proyecto**
LLM recibe solo código, error o tests.	LLM recibe también evidencia de análisis estático.
Se mide principalmente si el test pasa.	Se mide test passing, warnings, tamaño del patch y validez.
Se estudia reparación o detección por separado.	Se conecta detección estática con reparación automática.
El warning es tratado como salida final.	El warning se transforma en *repair evidence*.

Contribución esperada

Un estudio empírico reproducible que muestre si **static-analysis-guided prompting** mejora la reparación automática de software cuántico.

Objetivo general

Objetivo

Evaluar empíricamente si la evidencia producida por herramientas de análisis estático cuántico mejora la efectividad y la calidad de la reparación automática de bugs en programas cuánticos mediante LLMs.

Se busca medir:

- tasa de reparación;
- calidad del patch;
- reducción de warnings;
- categorías de bugs beneficiadas.

No se busca:

- crear un nuevo LLM;
- demostrar ventaja cuántica;
- ejecutar en hardware cuántico real;
- resolver todos los bugs posibles.

Preguntas de investigación

RQ1: Efectividad

¿Cómo afecta la evidencia de análisis estático cuántico la tasa de reparación de bugs mediante LLMs?

RQ2: Categorías de bugs

¿Qué tipos de bugs cuánticos se benefician más de la evidencia estática?

RQ3: Calidad del patch

¿La evidencia estática mejora la calidad del patch más allá de simplemente pasar los tests?

Hipótesis de trabajo

H1	Los prompts enriquecidos con evidencia estática tendrán mayor tasa de reparación que el baseline sin evidencia.
H2	La evidencia estática será más útil en bugs relacionados con estructura del circuito, qubits, gates y mediciones.
H3	Los parches generados con evidencia estática introducirán menos problemas detectables por análisis estático.

Importante

Si las hipótesis pueden no cumplirse. Un resultado negativo también puede ser publicable si el protocolo experimental es sólido.

Diseño experimental: cuatro configuraciones

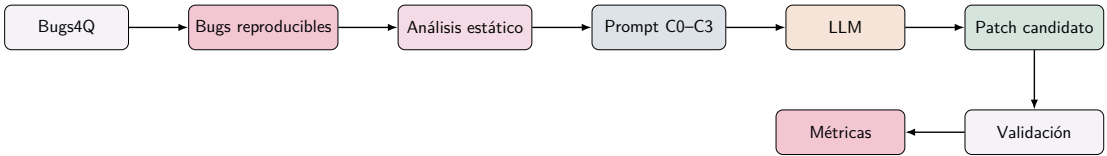
Config.	Nombre	Información entregada al LLM
C0	Baseline	Código buggy.
C1	Test-guided	Código buggy + test fallido + mensaje de error.
C2	Static-guided	Código buggy + warnings de análisis estático.
C3	Hybrid	Código buggy + test/error + warnings estáticos.

$$C_0 \quad vs. \quad C_1 \quad vs. \quad C_2 \quad vs. \quad C_3$$

Idea comparativa

Si C2 o C3 superan a C0/C1, entonces la evidencia estática estaría aportando valor real al proceso de reparación.

Pipeline del experimento



Todo debe quedar automatizado con scripts para poder repetir el experimento.

Dataset y alcance

Dataset base

- Bugs reales de Qiskit.
- Versiones buggy/fixed.
- Tests asociados.
- Preferir versión reproducible o ambiente fijado.

Alcance recomendado

- 25–35 bugs reproducibles.
- 1 LLM principal.
- 1 herramienta estática principal.
- 4 configuraciones de prompting.
- 3–5 intentos por bug/configuración.

Importante

Primero lograr un experimento pequeño y reproducible. Luego ampliar.

Qué cuenta como éxito

Nivel	Criterio	Interpretación
Compilación	El patch ejecuta sin error sintáctico o de importación.	Reparación mínima.
Patch plausible	El patch pasa los tests disponibles.	Podría ser correcto, pero no es suficiente.
Patch validado	Pasa tests, no introduce nuevos warnings críticos y es revisado contra la versión fixed.	Evidencia más fuerte de reparación.
Patch inválido	No compila, no pasa tests o cambia el comportamiento esperado.	Reparación fallida.

$$RepairRate = \frac{\#\text{patches validados}}{\#\text{bugs evaluados}}$$

Métricas principales

Métrica	Uso
Compilation rate	Ver si el LLM produce código ejecutable.
Test pass rate	Medir parches plausibles.
Validated repair rate	Medir reparaciones más confiables.
Warning reduction	Ver si el patch reduce problemas estáticos.
Introduced warnings	Detectar regresiones de calidad.
Patch size	Analizar si el patch es pequeño o invasivo.

MVP investigativo

Producto mínimo publicable

- Dataset curado de 25–35 bugs reproducibles.
- Pipeline que ejecuta C0, C1, C2 y C3.
- Uso de una herramienta estática principal.
- Validación por tests y comparación con fixed version.
- Resultados cuantitativos y análisis cualitativo de casos.
- Repositorio de replicación.

No agregar complejidad antes del MVP

No incluir varios LLMs, RAG, mutation analysis o hardware real antes de tener el pipeline funcionando.

Extensiones si el tiempo alcanza

Extensión	Valor científico	Riesgo
Segundo LLM	Permite comparar robustez entre modelos.	Aumenta costo y tiempo.
QChecker adicional	Mejora la evidencia estática.	Instalación/integración puede ser difícil.
RAG con documentación Qiskit	Ayuda a bugs de API y migración.	Requiere infraestructura adicional.
Mutation analysis	Permite validación más fuerte.	Puede ser caro experimentalmente.
Más bugs	Mejora generalización.	Puede afectar reproducibilidad.

Qué debe estudiar el estudiante

Área	Contenidos mínimos	Semanas
Qiskit básico	circuitos, qubits, gates, mediciones, simuladores.	1–2
Testing	unit tests, pytest, oráculos, test passing vs correctness.	2–4
Análisis estático	AST, linters, CodeQL, warnings, falsos positivos.	3–6
LLM para código	prompting, temperatura, reproducibilidad, evaluación de patches.	4–8
APR	bug, patch plausible, patch validado, overfitting de tests.	5–8
Método empírico	RQs, protocolo, métricas, amenazas a la validez.	5–19

Cronograma general de 19 semanas

Fase	Actividades	Salida
1–2	Fundamentos de Qiskit y programas cuánticos.	Ejercicios básicos.
3–4	Lectura de literatura: Bugs4Q, LintQ, LLM repair.	Mapa de literatura.
5–6	Definir protocolo, RQs, dataset y ambiente.	Protocolo experimental.
7–9	Implementar pipeline: tests, análisis estático y prompts.	Prototipo inicial.
10	Piloto con 5 bugs.	Decisión go/no-go.
11–13	Ejecutar experimento completo.	Datos experimentales.
14–15	Analizar resultados y casos representativos.	Tablas y figuras.
16–17	Redactar artículo/informe.	Manuscrito inicial.
18–19	Replication package, revisión y defensa.	Entrega final.

Hitos y entregables

Semana	Hito	Entregable
2	Comprensión inicial de Qiskit.	Notebook con 5 circuitos funcionando.
4	Estado del arte mínimo.	Tabla de 10–15 trabajos relacionados.
6	Protocolo cerrado.	Documento con RQs, métricas y dataset.
9	Pipeline inicial.	Script que procesa al menos 1 bug.
10	Piloto.	Resultados de 5 bugs en C0–C3.
13	Experimento completo.	CSV/JSON con resultados.
15	Análisis.	Figuras, tablas y hallazgos.
17	Manuscrito.	Paper/informe versión 1.
19	Cierre.	Paper final + repo + presentación.

Criterios de aceptación por hito

Hito	Criterio de aceptación
Dataset	Cada bug seleccionado compila o falla de forma controlada en el ambiente definido.
Análisis estático	El script registra warnings por bug y por configuración de forma estructurada.
Prompting	Todos los prompts C0–C3 se generan automáticamente desde plantillas.
Reparación	Cada patch queda guardado con identificador, configuración, bug y número de intento.
Validación	Para cada patch se registra compilación, tests, warnings y estado final.
Replicación	Otra persona puede ejecutar el pipeline con instrucciones claras.

Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Bugs no reproducibles	Reduce tamaño del dataset.	Usar ambiente fijado y seleccionar solo bugs reproducibles.
Instalar herramientas estáticas toma mucho tiempo	Retrasa el experimento.	Usar una herramienta principal y dejar otras como opcionales.
Costo o límite de API LLM	Experimento incompleto.	Reducir intentos por bug o usar modelo local/alternativo.
LLM genera parches incorrectos que pasan tests	Amenaza a validez.	Distinguir patch plausible y patch validado.
Demasiado alcance	No se termina.	Congelar MVP en semana 10.

Qué debería contener el repositorio

Código

- `scripts/`
- `prompts/`
- `experiments/`
- `analysis/`
- `notebooks/`

Datos y documentación

- `data/bugs.csv`
- `results/results.csv`
- `README.md`
- `environment.yml`
- `paper/`

Meta

Que el proyecto no sea solo un informe: debe dejar un paquete experimental reutilizable.

Resultados esperados

- Saber si la evidencia estática mejora la reparación frente al baseline.
- Identificar qué categorías de bugs se benefician más.
- Detectar cuándo la evidencia estática no ayuda o incluso confunde al LLM.
- Proponer recomendaciones para diseñar mejores herramientas de reparación cuántica.
- Generar un replication package para publicación.

Tipo de contribución

Método + pipeline + estudio empírico + datos replicables

Posible estructura del artículo

1. Introduction
2. Background: Quantum Software Engineering, Bugs4Q, static analysis, LLM repair
3. Related Work
4. Study Design
5. Experimental Setup
6. Results: RQ1–RQ3
7. Discussion
8. Threats to Validity
9. Conclusion

Clave para publicar

No venderlo como “el LLM repara bugs”, sino como evidencia empírica sobre **qué información de Ingeniería de Software mejora la reparación**.

Primeras tareas

1. Instalar Python, Qiskit, pytest y un ambiente reproducible.
2. Implementar circuitos simples: Bell, GHZ y teleportación básica.
3. Leer el paper/dataset de Bugs4Q.
4. Ejecutar al menos un bug de Bugs4Q.
5. Instalar o revisar una herramienta de análisis estático cuántico.
6. Diseñar la primera versión de los prompts C0–C3.

Meta de las primeras dos semanas

Que el estudiante pueda explicar con sus palabras qué es un bug cuántico, cómo se prueba y cómo se podría reparar.

Mensaje final

Este proyecto no trata de "usar IA para arreglar código".

Trata de investigar si la **evidencia de Ingeniería de Software** puede hacer más confiable la reparación automática de programas cuánticos.

Tu misión

Construir un experimento reproducible que permita responder, con datos, si el análisis estático ayuda a los LLMs a reparar mejor software cuántico.

Referencias iniciales

- Zhao et al. Bugs4Q: A benchmark of real bugs for quantum programs.
- LintQ: A static analysis framework for Qiskit programs. FSE 2024.
- QChecker: Detecting bugs in quantum programs via static analysis.
- The Smelly Eight: An empirical study on quantum code smells.
- LLM-based repair of quantum program bugs. Q-SE 2024.
- Mutation-guided LLM repair for quantum software. SANER 2026.

Estas referencias deben ampliarse durante las semanas 3–4 para construir el estado del arte del artículo.